# 📝 GLA UNIVERSITY — DAY 2 PRE-ASSESSMENT QUESTION PAPER

## Traditional Machine Learning & Parallel Computing using Intel oneAPI ( `GLA-HPC-BCSG701` )

**Time Allowed:** 45 Minutes | **Total Questions:** 50 Multi-Choice Questions | **Maximum Marks:** 50
**Candidate Name:** _____ | **Roll Number:** _____
**Date:** _____ | **Section:** _____

### Instructions to Candidates:

1. Each question carries 1 mark. There is no negative marking.
2. Select only ONE option (a, b, c, or d) for each question.

---

### Question 01 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** What is the primary difference between a process and a thread regarding their virtual memory address spaces in modern operating systems?

- **a)** Processes share heap memory by default, whereas threads have isolated heaps
- **b)** Threads within the same process share code, data, and heap segments but possess their own private stack and register state; processes have strictly isolated virtual address spaces
- **c)** Threads cannot execute CPU instructions simultaneously
- **d)** Processes do not require page tables

**Candidate Selection:** [ ]

---

### Question 02 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** When multiple threads concurrently execute `counter = counter + 1` without synchronization, why can the final value be smaller than expected?

- **a)** The CPU drops floating-point numbers
- **b)** The operation consists of three assembly instructions (Read, Modify, Write) that can be interleaved arbitrarily across threads, resulting in lost updates
- **c)** The OS scheduler deletes variables that grow too fast
- **d)** Threads run at different clock voltages

**Candidate Selection:** [ ]

---

### Question 03 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** What is the primary function of a Mutex (Mutual Exclusion Lock) in multi-threaded programming?

- **a)** To accelerate thread execution by 200%
- **b)** To ensure that only one thread can execute a critical section of code at any given time, blocking other threads until the lock is released
- **c)** To convert Python functions into assembly code
- **d)** To distribute network traffic across cluster switches

**Candidate Selection:** [ ]

---

### Question 04 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** Which of the following is NOT one of the four necessary Coffman conditions required for a system deadlock to occur?

- **a)** Mutual Exclusion
- **b)** Hold and Wait
- **c)** Preemption allowed by default
- **d)** Circular Wait

**Candidate Selection:** [ ]

---

### Question 05 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** Why is switching between two threads of the SAME process generally faster than switching between two threads of DIFFERENT processes?

- **a)** Same-process threads run on the same frequency
- **b)** Switching threads within the same process avoids flushing the Translation Lookaside Buffer (TLB) and switching the page table base register (CR3 on x86)
- **c)** Different processes must write their memory to hard disk before switching
- **d)** Same-process threads do not save CPU registers

**Candidate Selection:** [ ]

---

### Question 06 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** Under what specific condition is a Spinlock preferable over a standard blocking OS Mutex in high-performance computing?

- **a)** When the critical section is very long (takes several minutes)
- **b)** When the lock is held for an extremely short duration (a few nanoseconds/cycles) and the cost of an OS context switch exceeds the cost of busy-waiting
- **c)** When running on single-core uniprocessor systems
- **d)** When writing data to remote tape storage

**Candidate Selection:** [ ]

---

### Question 07 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** What is the typical physical size of an L1 Data Cache line on modern Intel Xeon and x86_64 processors?

- **a)** 8 bytes
- **b)** 64 bytes
- **c)** 512 bytes
- **d)** 4096 bytes

**Candidate Selection:** [ ]

---

### Question 08 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** In C and C++, 2D arrays are stored in Row-Major order in memory. Which nested loop traversal yields the highest cache hit rate for an $N \times N$ matrix?

- **a)** `for (j=0; j<N; j++) for (i=0; i<N; i++) A[i][j] = 0;` (Column-by-Column)
- **b)** `for (i=0; i<N; i++) for (j=0; j<N; j++) A[i][j] = 0;` (Row-by-Row)
- **c)** Both loops have identical cache hit rates
- **d)** Random index hopping

**Candidate Selection:** [ ]

---

### Question 09 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** What does declaring a variable `volatile int flag;` instruct the C/C++ compiler to do during optimization?

- **a)** It makes all operations on `flag` atomic across threads
- **b)** It prevents the compiler from caching `flag` in a CPU register, forcing every read and write to access physical memory/cache directly
- **c)** It places the variable in the GPU memory space
- **d)** It automatically encrypts the variable value

**Candidate Selection:** [ ]

---

### Question 10 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** Why are Memory Barriers (fences) necessary when implementing lock-free data structures on modern out-of-order execution CPUs?

- **a)** To stop the CPU fan from spinning too fast
- **b)** To enforce strict memory ordering, preventing hardware CPU pipelines and optimizing compilers from reordering reads and writes across the barrier boundary
- **c)** To convert 32-bit pointers to 64-bit pointers
- **d)** To force the operating system to shut down idle cores

**Candidate Selection:** [ ]

---

### Question 11 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** Which IPC mechanism on a single Linux host provides the highest data transfer bandwidth by eliminating kernel data copying?

- **a)** UNIX domain sockets
- **b)** POSIX Shared Memory ( `shm_open` / `mmap` )
- **c)** Anonymous Pipes ( `pipe()` )
- **d)** Standard TCP loopback sockets ( `127.0.0.1` )

**Candidate Selection:** [ ]

---

### Question 12 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** In C POSIX threads ( `pthread` ), which function is called by a parent thread to block until a specific target child thread has completed execution?

- **a)** `pthread_yield()`
- **b)** `pthread_join()`
- **c)** `pthread_exit()`
- **d)** `pthread_detach()`

**Candidate Selection:** [ ]

---

### Question 13 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** Why do high-performance servers and OpenMP runtimes utilize a pre-allocated 'Thread Pool' rather than creating a new thread for every incoming task?

- **a)** Because operating systems can only create a maximum of 4 threads in their lifetime
- **b)** To eliminate the substantial kernel latency of repeatedly allocating thread stacks, creating kernel scheduling entities, and deallocating thread resources on every request
- **c)** To ensure that all tasks execute in alphabetical order
- **d)** Because thread pools eliminate the need for RAM

**Candidate Selection:** [ ]

---

### Question 14 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** What does it mean for a C++ library function (e.g., in a math library) to be 'Thread-Safe'?

- **a)** It can only be executed by one single thread across the entire program runtime
- **b)** It can be invoked simultaneously by multiple threads without causing race conditions, memory corruption, or undefined behavior
- **c)** It automatically disables multi-threading in the calling program
- **d)** It requires root permissions to execute

**Candidate Selection:** [ ]

---

### Question 15 [Prerequisite Foundations: Operating Systems & Concurrency]

**Question:** What is the classical formula for Amdahl's Law calculating speedup $S(N)$ for a parallel fraction $P$ and sequential fraction $S = 1 - P$ on $N$ processors?

- **a)** $S(N) = N \times P$
- **b)** $S(N) = \frac{1}{S + \frac{P}{N}}$
- **c)** $S(N) = \frac{1}{N} + P$
- **d)** $S(N) = P - S \times N$

**Candidate Selection:** [ ]

---

### Question 16 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** What is the fundamental distinguishing characteristic of Supervised Learning compared to Unsupervised Learning?

- **a)** Supervised learning requires GPU acceleration, while unsupervised does not
- **b)** Supervised learning trains on labeled input-output pairs $(X, y)$ to learn a mapping function; Unsupervised learning discovers latent patterns and clusters within unlabeled data $X$ without ground-truth targets
- **c)** Supervised learning only works on images, while unsupervised works on text
- **d)** Unsupervised learning always achieves 100% accuracy

**Candidate Selection:** [ ]

---

### Question 17 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** What mathematical objective function does the classical K-Means algorithm minimize during iterative centroid updates?

- **a)** Cross-entropy loss
- **b)** Within-Cluster Sum of Squares (WCSS) / Inertia, measuring the sum of squared Euclidean distances between points and their assigned cluster centroids
- **c)** Hinge loss
- **d)** Maximum likelihood of Markov chains

**Candidate Selection:** [ ]

---

### Question 18 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** What is the mathematical goal of Principal Component Analysis (PCA) in machine learning preprocessing?

- **a)** To convert tabular data into audio waveforms
- **b)** To project high-dimensional data onto orthogonal axes (eigenvectors of the covariance matrix) that maximize the variance of the data along the principal dimensions
- **c)** To eliminate rows with negative numbers
- **d)** To encrypt user data for privacy compliance

**Candidate Selection:** [ ]

---

### Question 19 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** In a linear Support Vector Classifier (SVC), what defines the 'optimal margin' separating two classes?

- **a)** The line that connects the average of all points in both classes
- **b)** The decision boundary that maximizes the geometric distance (margin) between the separating hyperplane and the closest data points (support vectors) of either class
- **c)** The curve that passes through every single data point with zero residual
- **d)** The boundary chosen randomly by gradient descent

**Candidate Selection:** [ ]

---

### Question 20 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** How does a Random Forest Classifier improve predictive accuracy and prevent overfitting compared to a single Decision Tree?

- **a)** By running decision trees in reverse order
- **b)** By training an ensemble of de-correlated decision trees using bootstrap aggregating (bagging) and random feature subsets, then averaging their predictions (majority voting)
- **c)** By deleting all trees that have more than 3 branches
- **d)** By converting all input features into binary zeros and ones

**Candidate Selection:** [ ]

---

### Question 21 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** An ML model achieves 99.8% training accuracy on a cluster dataset but drops to 52.1% accuracy on the independent test validation set. What is this phenomenon called?

- **a)** Underfitting (High Bias)
- **b)** Overfitting (High Variance)
- **c)** Optimal Generalization
- **d)** Data Quantization

**Candidate Selection:** [ ]

---

### Question 22 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** Why is feature standardization (`StandardScaler` in Scikit-Learn) mandatory before training distance-based algorithms like K-Means, KNN, and SVM with RBF kernels?

- **a)** Because Python cannot divide by numbers greater than 100
- **b)** Because features with large numerical magnitudes (e.g., annual income in thousands) would dominate Euclidean distance calculations over features with small scales (e.g., age in years)
- **c)** To convert continuous numbers into text strings
- **d)** StandardScaler is only used to format plots for publication

**Candidate Selection:** [ ]

---

### Question 23 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** In Scikit-Learn's object-oriented API design, which method is consistently implemented across all estimator algorithms to execute model training?

- **a)** `model.train()`
- **b)** `model.fit(X, y)`
- **c)** `model.learn()`
- **d)** `model.execute()`

**Candidate Selection:** [ ]

---

### Question 24 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** Why do high-performance C/C++ libraries (like oneDAL and BLAS) execute significantly faster on `C_CONTIGUOUS` NumPy arrays compared to non-contiguous sliced arrays?

- **a)** Non-contiguous arrays can only hold integer numbers
- **b)** `C_CONTIGUOUS` arrays reside in a single flat block of linear physical memory where consecutive row elements are adjacent, enabling direct vector SIMD memory loads without pointer indirection or memory copying
- **c)** NumPy non-contiguous arrays are stored on hard disk instead of RAM
- **d)** Python prohibits non-contiguous array indexing

**Candidate Selection:** [ ]

---

### Question 25 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** How does the Python Global Interpreter Lock (GIL) impact pure Python multi-threaded CPU-bound programs on multi-core servers?

- **a)** It doubles the speed of Python loops on multiple cores
- **b)** It prevents multiple OS threads from executing Python bytecode simultaneously, effectively serializing execution onto a single CPU core despite having multiple threads
- **c)** It forces Python to run inside a web browser
- **d)** It automatically converts Python to C++ code

**Candidate Selection:** [ ]

---

### Question 26 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** Why do NumPy vector operations (e.g., `np.dot(A, B)`) and Intel oneDAL kernels scale across multiple CPU cores in Python despite the existence of the GIL?

- **a)** NumPy runs in a separate Linux container
- **b)** C/C++ extensions and oneDAL explicitly release the Python GIL (`Py_BEGIN_ALLOW_THREADS`) before entering compute-intensive loops, allowing native threads to execute concurrently across physical cores
- **c)** NumPy deletes the GIL permanently upon installation
- **d)** NumPy only uses 1 core at all times

**Candidate Selection:** [ ]

### Question 27 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** What fatal methodological flaw occurs if a data scientist fits `StandardScaler().fit(X)` on the ENTIRE dataset before performing `train_test_split()`?

- **a)** The computer runs out of memory
- **b)** Data Leakage: Information from the test validation set (such as the mean and standard deviation) leaks into the training pipeline, yielding overly optimistic evaluation metrics that fail in production
- **c)** Scikit-learn raises a `SyntaxError` exception
- **d)** The test set becomes empty

**Candidate Selection:** [ ]

---

### Question 28 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** In medical disease diagnosis or critical fraud detection where false negatives (missing a true positive condition) have severe consequences, which metric should be prioritized?

- **a)** Accuracy on an imbalanced dataset
- **b)** Recall (Sensitivity / True Positive Rate)
- **c)** Training Loss
- **d)** Number of features

**Candidate Selection:** [ ]

---

### Question 29 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** What mathematical function does Logistic Regression apply to linear combinations of inputs ($z = w^T x + b$) to map outputs to probabilities between 0 and 1?

- **a)** Rectified Linear Unit (ReLU)
- **b)** Sigmoid (Logistic) Function: $\sigma(z) = \frac{1}{1+e^{-z}}$
- **c)** Hyperbolic Tangent ($tanh$)
- **d)** Cosine similarity

**Candidate Selection:** [ ]

---

### Question 30 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn]

**Question:** How does 5-Fold Cross-Validation provide a more robust performance estimate than a single train/test split?

- **a)** It generates synthetic data using deep generative models
- **b)** It partitions the dataset into 5 equal folds, training 5 separate models where each fold serves as the test set once while the remaining 4 serve as training, averaging the validation scores
- **c)** It runs the training loop 5 times on the exact same data to verify Python speed
- **d)** It divides the dataset into 5 separate CSV files on disk

**Candidate Selection:** [ ]

---

## Question 31 [Applied Parallel Concepts & Architecture Preview]

**Question:** A cluster node contains an Intel Xeon CPU executing multiple independent MPI processes simultaneously, while its vector registers execute AVX-512 vector instructions. Under Flynn's taxonomy, how is this system categorized at the node and core levels respectively?

- **a)** SISD at the node level, MISD at the core level
- **b)** MIMD (Multiple Instruction, Multiple Data) across nodes/processes, and SIMD (Single Instruction, Multiple Data) within individual vector cores
- **c)** MISD across nodes, SISD within cores
- **d)** Pure SISD exclusively throughout the cluster

**Candidate Selection:** [ ]

---

## Question 32 [Applied Parallel Concepts & Architecture Preview]

**Question:** What is the defining distinction between 'Strong Scaling' and 'Weak Scaling' in parallel performance benchmarking?

- **a)** Strong scaling uses Intel CPUs; Weak scaling uses AMD CPUs
- **b)** In Strong Scaling, the total problem size remains fixed as processor count increases; in Weak Scaling, the problem size increases proportionally with the number of processors
- **c)** Strong scaling measures GPU speed; Weak scaling measures disk speed
- **d)** Weak scaling assumes all processors run at half voltage

**Candidate Selection:** [ ]

---

## Question 33 [Applied Parallel Concepts & Architecture Preview]

**Question:** Why is inter-node network latency (e.g., Ethernet vs InfiniBand) often the dominant bottleneck in distributed MPI training rather than pure CPU FLOPS?

- **a)** Because CPUs execute code too slowly to notice the network
- **b)** Because network transfer latency (microseconds to milliseconds) is orders of magnitude slower than on-chip L1/L2 cache accesses (nanoseconds), causing processors to stall waiting for data transfers
- **c)** Because network cables compress data automatically
- **d)** Because Linux terminates network sockets every 10 seconds

**Candidate Selection:** [ ]

---

## Question 34 [Applied Parallel Concepts & Architecture Preview]

**Question:** Can OpenMP parallel directives (`#pragma omp parallel`) run across multiple separate server computers connected via network cables without MPI?

- **a)** Yes, OpenMP automatically creates a virtual cloud across computers
- **b)** No, OpenMP is strictly designed for Shared-Memory systems where threads access the same physical motherboard address space
- **c)** Yes, but only if all computers run Windows 11
- **d)** Yes, if using USB cables between motherboards

**Candidate Selection:** [ ]

---

## Question 35 [Applied Parallel Concepts & Architecture Preview]

**Question:** When four MPI ranks run on the same physical dual-socket server, how do their memory address spaces interact by default?

- **a)** They share all global variables automatically
- **b)** Each MPI rank is an independent operating system process with its own private virtual memory space; data sharing requires explicit MPI calls or shared-memory IPC
- **c)** They merge into a single thread
- **d)** They can write to each other's stack pointers without restrictions

**Candidate Selection:** [ ]

---

## Question 36 [Applied Parallel Concepts & Architecture Preview]

**Question:** What is the industry-standard 'Hybrid Programming Model' widely deployed on top supercomputers today?

- **a)** Python for front-end, HTML for back-end
- **b)** MPI across distributed nodes (inter-node communication) combined with OpenMP multithreading within each shared-memory node (intra-node core parallelism)
- **c)** JavaScript running inside bash scripts
- **d)** Java Virtual Machines communicating over Bluetooth

**Candidate Selection:** [ ]

---

## Question 37 [Applied Parallel Concepts & Architecture Preview]

**Question:** What happens if a parallel algorithm has 'Fine-Grained' parallelism where the computation per task is extremely small compared to thread synchronization overhead?

- **a)** The program achieves linear super-speedup
- **b)** Parallel overhead (thread creation, lock contention, scheduling) exceeds the computational workload, causing the parallel program to run slower than the sequential version
- **c)** The CPU frequency doubles
- **d)** The program crashes with an out-of-memory error

**Candidate Selection:** [ ]

---

## Question 38 [Applied Parallel Concepts & Architecture Preview]

**Question:** In a parallel computing cluster, what is 'Load Imbalance'?

- **a)** When the server rack is physically tilted on the floor
- **b)** When unequal amounts of computational work are assigned to different processors, forcing fast workers to sit idle at barriers waiting for the slowest worker to finish
- **c)** When the power supply provides unbalanced electrical voltage
- **d)** When all cores complete work at the exact same microsecond

**Candidate Selection:** [ ]

---

## Question 39 [Applied Parallel Concepts & Architecture Preview]

**Question:** Which of the following loops CANNOT be safely parallelized with OpenMP `#pragma omp parallel for` without restructuring?

- **a)** `for (int i=0; i<N; i++) C[i] = A[i] + B[i];`
- **b)** `for (int i=1; i<N; i++) A[i] = A[i-1] + B[i];` (Read-After-Write Loop-Carried Dependency)
- **c)** `for (int i=0; i<N; i++) A[i] = 2.0 * i;`
- **d)** `for (int i=0; i<N; i++) C[i] = sin(A[i]);`

**Candidate Selection:** [ ]

---

## Question 40 [Applied Parallel Concepts & Architecture Preview]

**Question:** When running a parallel code across 1, 2, 4, 8, 16, and 32 cores, you notice execution time decreases up to 8 cores, but increases when going from 16 to 32 cores. What is the most common hardware root cause?

- **a)** The CPU ran out of clock cycles
- **b)** Memory Bandwidth Saturation (all cores saturate DDR memory channels) and lock contention/inter-socket NUMA bus overhead outweigh compute gains
- **c)** Linux automatically limits programs to 8 cores
- **d)** The C++ compiler disables optimization past 8 cores

**Candidate Selection:** [ ]

---

## Question 41 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** Which family of machine learning algorithms is accelerated by the Intel oneAPI Data Analytics Library (oneDAL)?

- **a)** Deep Convolutional Neural Networks only
- **b)** Classical Machine Learning and Data Analytics (K-Means, SVM, PCA, Linear/Logistic Regression, Random Forest, KNN)
- **c)** Large Language Model text generation exclusively
- **d)** Quantum circuit simulation

**Candidate Selection:** [ ]

---

## Question 42 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** What is the official package name for the Intel Extension for Scikit-Learn available on PyPI and Conda?

- **a)** `intel-scikit`
- **b)** `scikit-learn-intelex`
- **c)** `fast-sklearn-intel`
- **d)** `mkl-learn`

**Candidate Selection:** [ ]

## Question 43 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** Which low-level Intel mathematics library powers matrix algebra operations inside Intel-optimized NumPy distributions?

- **a)** Intel oneMKL (Math Kernel Library)
- **b)** OpenCV
- **c)** DirectX
- **d)** FFmpeg

**Candidate Selection:** [ ]

---

## Question 44 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** How does Modin (included in the Intel AI Analytics Toolkit) accelerate standard Pandas DataFrame operations?

- **a)** By deleting 50% of the data columns
- **b)** By partitioning DataFrames across both rows and columns and distributing computations seamlessly across all available CPU cores using Ray or Dask backends
- **c)** By converting all numbers into integers
- **d)** By running dataframes on GPU ray tracing cores

**Candidate Selection:** [ ]

---

## Question 45 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** Which compiler flag must be passed to `g++` or `icpx` to enable OpenMP directive parsing and link the OpenMP runtime library?

- **a)** `-open-all`
- **b)** `-fopenmp` (for GCC/Clang) or `-qopenmp` (for Intel icx/icpx)
- **c)** `-parallel-mode`
- **d)** `-omp-enable`

**Candidate Selection:** [ ]

---

## Question 46 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** Which command-line wrapper script is used to compile C++ MPI applications with all required headers and libraries automatically linked?

- **a)** `g++ -mpi`
- **b)** `mpicxx` (or `mpic++`)
- **c)** `compile-mpi`
- **d)** `mpi-build`

**Candidate Selection:** [ ]

---

### Question 47 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** In OpenMP multithreading, why does an array `int counter[NUM_THREADS]` where each thread `tid` increments `counter[tid]` suffer from severe performance degradation?

- **a)** Because the array exceeds total RAM capacity
- **b)** Because all elements of `counter` sit within the same 64-byte L1 cache line, causing constant false sharing and MESI cache invalidation traffic between CPU cores
- **c)** Because OpenMP cannot index arrays using thread IDs
- **d)** Because C++ integers are not thread-safe

**Candidate Selection:** [ ]

---

### Question 48 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** Why should `#pragma omp critical` be placed OUTSIDE high-frequency inner loops whenever possible?

- **a)** Because critical sections disable internet access
- **b)** Because entering and exiting a critical section involves mutex lock acquisitions that serialize all threads, completely eliminating parallel speedup if executed millions of times
- **c)** Because critical sections can only execute 10 times total
- **d)** Because the compiler deletes critical sections during optimization

**Candidate Selection:** [ ]

---

### Question 49 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** Why is replacing a loop of $N$ point-to-point `MPI_Send` calls with a collective `MPI_Bcast` dramatically faster on large HPC clusters?

- **a)** `MPI_Bcast` runs on the motherboard audio card
- **b)** `MPI_Bcast` utilizes hardware-optimized binomial or hypercube tree algorithms with $O(\log P)$ communication steps, whereas a loop of sends takes $O(P)$ sequential steps bottlenecked at Rank 0
- **c)** `MPI_Send` cannot transmit arrays larger than 100 bytes
- **d)** `MPI_Bcast` disables network switches

**Candidate Selection:** [ ]

---

### Question 50 [Day 2 Advanced Architecture & oneDAL Preview]

**Question:** When executing `sklearnex.patch_sklearn()`, which hardware instruction set extensions on 4th and 5th Gen Intel Xeon Scalable processors does oneDAL leverage for maximum throughput?

- **a)** ARM Neon instructions
- **b)** Intel AVX-512 vector extensions and Intel AMX (Advanced Matrix Extensions)
- **c)** Legacy MMX 64-bit integer registers
- **d)** 8086 16-bit real mode registers

**Candidate Selection:** [ ]

---